

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: K.Vinay

Roll No.: A24126552086

Date: 31-08-2026

1. TITLE

Development of an Interactive To-Do List Application using HTML, CSS, and DOM Manipulation

2. AIM

To create a functional to-do list web application that allows users to add tasks, mark them as complete, and delete them dynamically using JavaScript Event Listeners and DOM manipulation.

3. OBJECTIVE

The objectives of this experiment are:

- To design a clean user interface for task entry using HTML and CSS.
 \\n
- To access DOM elements using getElementById.
 \\n
- To attach event listeners to buttons for user interaction.
 \\n
- To dynamically create, append, and remove HTML elements using JavaScript.
 \\n
- To toggle CSS classes for visual state changes.

4. THEORY

JavaScript allows dynamic updates to a web page by interacting with the Document Object Model (DOM). Elements can be created (document.createElement), modified (innerText, className), added (appendChild), or removed (remove()) in response to user events such as button clicks.

5. ALGORITHM / PROCEDURE

1. Create the UI layout with an input field, an Add button, and a list container.
 \\n
2. Style the components using CSS for a clean, centered layout.
 \\n
3. Select required elements in JavaScript (input, add, list, message).

- \n
4. Add a click event listener to the Add button.
 - \n
 5. On click, validate the input and dynamically create a div for the new task.
 - \n
 6. Create and attach Complete and Delete buttons to the task item.
 - \n
 7. Attach onclick events to toggle strikethrough and remove elements.
 - \n
 8. Append the new task element to the list container and clear the input field.

6. PROGRAM

index.html

```

<!DOCTYPE html>
<html>
<head>
  <title>To do list</title>

  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #e8f5e9;
      text-align: center;
      padding: 40px;
    }

    .box {
      width: 450px;
      margin: auto;
      padding: 25px;
      background-color: white;
      border-radius: 12px;
      box-shadow: 0 0 10px #999;
    }

    h1 {
      color: #2e7d32;
    }

    input {
      width: 260px;
      padding: 10px;
      border: 1px solid #aaa;
      border-radius: 5px;
    }

    #add {
      padding: 10px 15px;
      background-color: #2e7d32;
      color: white;
      border: none;
      border-radius: 5px;
      cursor: pointer;
    }

    .item {
      display: flex;
      align-items: center;
      margin-top: 12px;
      padding: 10px;
      background-color: #f1f8e9;
      border-radius: 6px;
    }

    .item span {
      flex: 1;

```

```

        text-align: left;
    }

    .done {
        text-decoration: line-through;
        color: gray;
    }

    .complete {
        background-color: #ff9800;
        color: white;
        border: none;
        padding: 8px;
        border-radius: 4px;
        cursor: pointer;
    }

    .remove {
        background-color: #f44336;
        color: white;
        border: none;
        padding: 8px;
        margin-left: 5px;
        border-radius: 4px;
        cursor: pointer;
    }
}
</style>
</head>

<body>

    <div class="box">

        <h1>To do list</h1>

        <input type="text" id="task" placeholder="Enter your task">

        <button id="add">Add</button>

        <p id="message">No tasks available.</p>

        <div id="list"></div>

    </div>

    <script>

        let input = document.getElementById("task");
        let add = document.getElementById("add");
        let list = document.getElementById("list");
        let message = document.getElementById("message");

```

```

add.addEventListener("click", function() {

    let value = input.value.trim();

    if (value === "") {
        alert("Enter a task first!");
        return;
    }

    // Create task item
    let item = document.createElement("div");
    item.className = "item";

    // Task name
    let taskName = document.createElement("span");
    taskName.innerText = value;

    // Complete button
    let complete = document.createElement("button");
    complete.innerText = "Complete";
    complete.className = "complete";

    complete.onclick = function() {
        taskName.classList.toggle("done");
    };

    // Delete button
    let remove = document.createElement("button");
    remove.innerText = "Delete";
    remove.className = "remove";

    remove.onclick = function() {
        item.remove();

        if (list.children.length === 0) {
            message.style.display = "block";
        }
    };

    // Add elements
    item.appendChild(taskName);
    item.appendChild(complete);
    item.appendChild(remove);

    list.appendChild(item);

    // Hide empty message
    message.style.display = "none";

    // Clear input
    input.value = "";

```

```

    });

</script>

</body>
</html>

```

7. CODE EXPLANATION

Code / Syntax	Explanation
document.createElement()	Creates a new element dynamically.
addEventListener('click')	Attaches a click event handler to an element.
appendChild(item)	Adds the newly created task item to the container.
classList.toggle('done')	Toggles the 'done' class to apply strikethrough styling.
remove()	Removes the selected element from the DOM.

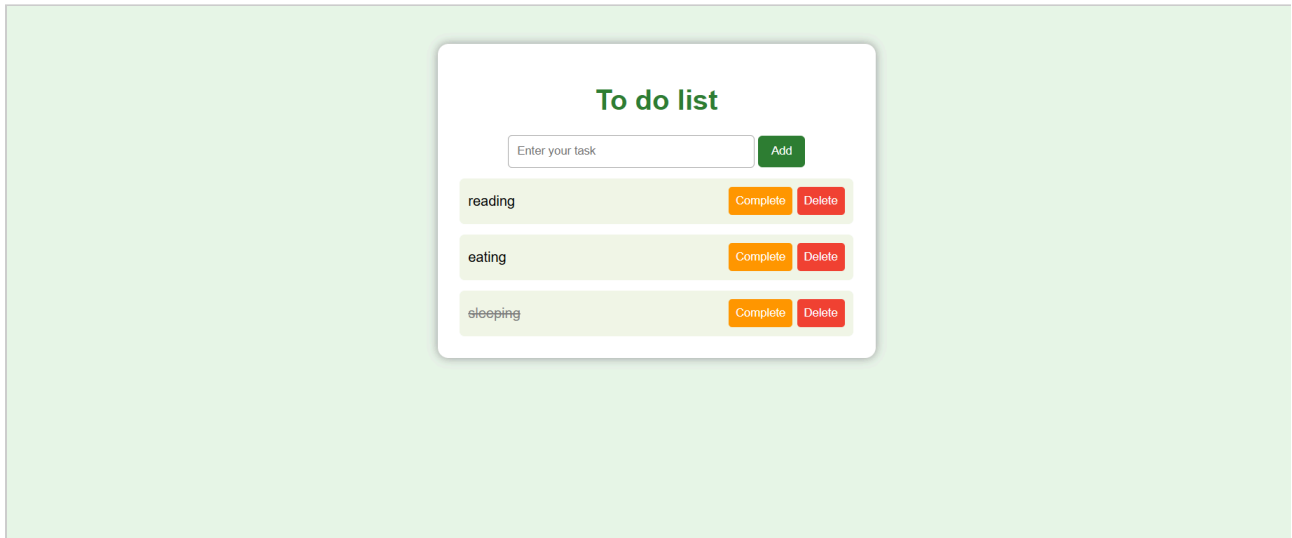
8. TEST CASES AND EXECUTION DATA

The experiment was executed with the following test cases to verify functionality:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Add Task	Enter task and click Add	Task is appended to the list	Pass
TC-02	Complete Task	Click Complete button	Strikethrough style is applied	Pass
TC-03	Delete Task	Click Delete button	Task is removed from the DOM	Pass

9. OUTPUT

Output 1: Development of an Interactive To-Do List Application using HTML, CSS, and DOM Manipulation



10. RESULT

Thus, an interactive To-Do List application was successfully developed, demonstrating effective DOM manipulation and event handling to manage dynamic task creation, completion, and deletion.

Full Stack Web Development Lab (23CM4121) | K.Vinay - A24126552086